

Guía del alumno

Módulo 5, Aprendizaje no supervisado

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después de la clase. Trae el mismo código, las mismas cifras y los mismos gráficos que viste en el laboratorio, con la explicación escrita y la solución de cada ejercicio.

Este módulo se dicta en dos clases. El bloque 1 es de la primera y los bloques 2 a 7 de la segunda. Cuando vuelvas a abrir el notebook, ejecútalo desde arriba, porque cada bloque usa lo que armó el anterior.

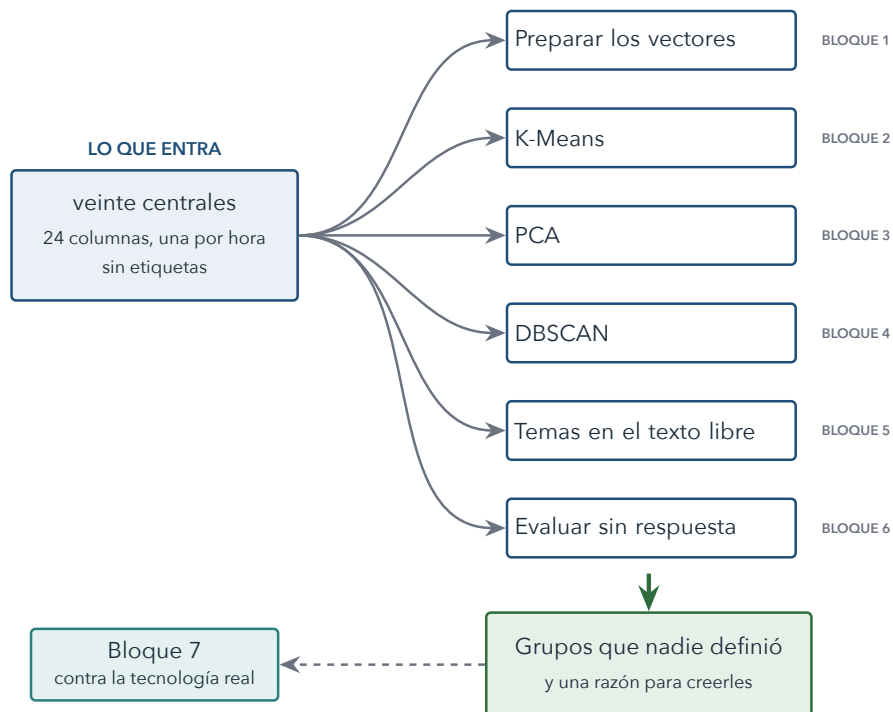
Lo que vas a poder hacer al terminar

- Preparar una tabla para agrupar, con las dos normalizaciones que hacen falta.
- Agrupar con K-Means y justificar el número de grupos con el codo y la silueta.
- Reducir veinticuatro columnas a dos con PCA y dibujar el resultado.
- Separar grupos densos de ruido con DBSCAN, y entender qué significa ese ruido.
- Encontrar temas en texto libre que nadie etiquetó antes.
- Defender un agrupamiento cuando no existe una respuesta correcta.

Todas las cifras y los gráficos de esta guía salen de la corrida verificada del notebook. Los datos se generan dentro del notebook con semilla fija, así que tus resultados son los mismos. Los datos son sintéticos, y donde una cifra del laboratorio se aleja de la realidad del sistema chileno, la guía lo dice.

1. El módulo en una imagen

Entra una tabla de veinte centrales por veinticuatro horas, sin etiquetas. Salen grupos que nadie definió, y una razón para creerles.



La imagen de la bodega sin inventario

Llegas a una bodega que lleva años funcionando y nadie hizo nunca un inventario. Hay miles de piezas en estantes, sin etiqueta, sin código, sin categoría. El bodeguero que sabía dónde estaba todo ya no trabaja acá.

No te van a decir cuántas familias de piezas hay. No hay una lista contra la cual comparar. Lo único que puedes hacer es mirar las piezas y juntar las que se parecen.

Y ahí aparece la pregunta que manda. Parecidas en qué. Por tamaño, por material, por para qué sirven. Cada respuesta da un inventario distinto y todos son defendibles.

Eso es aprendizaje no supervisado. No hay respuesta correcta, hay una decisión bien tomada y bien justificada, o una mal tomada que nadie revisó.

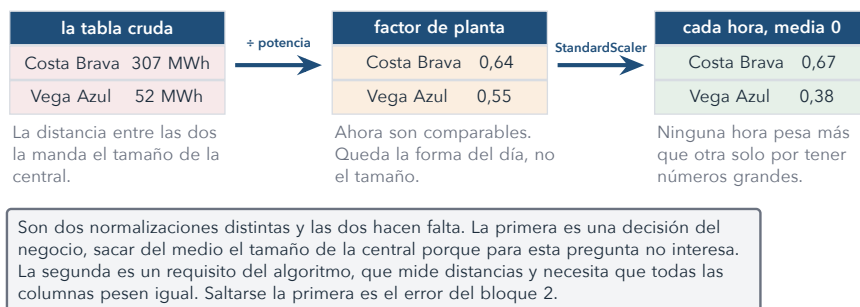
Lo que este módulo no hace

No predice. Describe. Ninguno de los cinco métodos de este módulo estima un valor futuro ni se puede evaluar con un error. Si llegaste esperando predicción, eso es otra familia de métodos.

2. Bloque 1, preparar los vectores

Qué se vio

Este bloque parece preparación y es el más importante del módulo. Todo lo que viene después mide distancias, y una distancia sobre datos mal preparados mide lo que no era.



El código, paso a paso

1.1 El perfil

```
import pandas as pd

# Cada central como un vector de 24 números, su día promedio. Igual que en el Lab 04.
gen = pd.read_csv("generacion.csv", parse_dates=["fecha"])
centrales = pd.read_csv("centrales.csv")
perfil = gen.pivot_table(index="central", columns="hora", values="mwh", aggfunc="mean")

print(perfil.shape)
print(perfil.iloc[:3, 10:15].round(1))
```

```
(20, 24)
hora          10    11    12    13    14
central
Carboeléctrica Costa Brava  306.8  307.2  307.4  306.8  307.3
Carboeléctrica Roca Gris    223.9  224.3  224.0  224.2  224.1
Central Aguas Claras        98.9   99.0   98.7   98.8   98.9
```

Veinte centrales, veinticuatro columnas. Cada fila es lo que se llama un vector de veinticuatro dimensiones, aunque suene grande, es esta tabla.

1.2 El problema del tamaño

```
# El problema de siempre. La Costa Brava genera 300 MWh a las 12 y la Vega Azul 50.
print(perfil.max(axis=1).sort_values(ascending=False).round(1).to_string())
```

```
central
Carboeléctrica Costa Brava      307.9
Termoeléctrica Bahía Norte      249.5
Central Río Manso Alto          232.9
Carboeléctrica Roca Gris        224.3
Termoeléctrica Puerto Sur       192.8
Central Salto Verde             171.3
Central Tres Saltos             143.7
Termoeléctrica Valle Central    139.9
Central Aguas Claras            99.3
Parque Solar Altiplano          81.9
Eólica Campo Abierto            65.7
... [y sigue hasta la más chica]
```

Trescientos ocho megawatt hora la más grande. Si mides distancias sobre estos números, las centrales se van a agrupar por tamaño y no por forma.

1.3 La primera normalización

```
# Primera normalización, dividir por la potencia instalada. Ahora cada número
# es un factor de planta horario, entre cero y uno, y todas son comparables.
potencia = centrales.set_index("central")["potencia_mw"]
factor = perfil.div(potencia, axis=0)

print("antes, rango de la tabla ", round(perfil.values.min(), 1), "a",
      round(perfil.values.max(), 1))
print("después, rango de la tabla", round(factor.values.min(), 3), "a",
      round(factor.values.max(), 3))
```

```
antes, rango de la tabla  0.0 a 307.9
después, rango de la tabla 0.0 a 0.666
```

Cada fila se divide por la potencia instalada de esa central. El número que queda tiene nombre propio, es el factor de planta de esa hora, va entre cero y uno y es una cifra que la gente del sector usa todos los días.

1.4 Cómo se ve normalizado

```
# Así se ve una central después de normalizar. Es su forma, no su tamaño.  
print(factor.loc["Parque Solar Altiplano"].round(2).to_string())
```

```
hora  
0      0.00  
1      0.00  
2      0.00  
3      0.00  
4      0.00  
5      0.00  
6      0.01  
7      0.05  
8      0.11  
9      0.16  
10     0.21  
11     0.25  
... [y el resto del día en cero]
```

Un parque solar. Cero toda la noche, sube desde las siete y 0,26 al mediodía. Es la forma del día, sin el tamaño de la central adentro.

1.5 La segunda normalización

```
from sklearn.preprocessing import StandardScaler  
  
# Segunda normalización, la de la biblioteca. Deja cada columna con promedio  
# cero y desviación uno, que es lo que esperan casi todos estos algoritmos.  
escalador = StandardScaler()  
X = escalador.fit_transform(factor)  
  
print(X.shape)  
print("promedio de X", round(float(X.mean()), 6), " desviación", round(float(X.std()), 3))
```

```
(20, 24)  
promedio de X -0.0 desviación 1.0
```

El StandardScaler deja cada columna con promedio cero y desviación uno. Sin esto, las horas del mediodía pesarían más en la distancia solo porque sus números son más grandes.

La solución del ejercicio

El ejercicio pedía comparar la central más grande con una de las más chicas.

```
dos = ["Carboeléctrica Costa Brava", "Central Vega Azul"]

print(factor.loc[dos, 10:14].round(2).to_string())
```

En crudo son 306,8 contra 52,0, o sea seis veces. Normalizadas quedan en 0,64 y 0,55, que es casi lo mismo, porque las dos son centrales de base que generan parejo todo el día. La diferencia era el tamaño, no la forma.

DOS NORMALIZACIONES DISTINTAS, Y LAS DOS HACEN FALTA

Dividir por la potencia instalada es una decisión del negocio. Saca del medio el tamaño de la central, que para esta pregunta no interesa, y deja el factor de planta, que es comparable entre una central de 480 MW y una de 45.

El StandardScaler es una decisión técnica. Deja cada hora con promedio cero y desviación uno, para que ninguna hora pese más que otra solo porque sus números son más grandes.

Son cosas distintas y las dos se hacen. En el bloque 2 se ve qué pasa si uno se salta la primera.

Referencia rápida

Instrucción	Para qué sirve
<code>df.pivot_table(index=, columns=)</code>	Una fila por entidad, una columna por dimensión
<code>df.div(serie, axis=0)</code>	Dividir cada fila por un valor propio
<code>df.div(df.max(axis=1), axis=0)</code>	Normalizar cada fila por su propio máximo
<code>StandardScaler().fit_transform(df)</code>	Cada columna con promedio 0 y desviación 1
<code>MinMaxScaler()</code>	Lleva cada columna al rango 0 a 1
<code>RobustScaler()</code>	Usa mediana y rango intercuartílico, aguanta extremos

Errores comunes

Lo que aparece en pantalla	Qué pasó
Los grupos quedan por tamaño	Falta dividir por la potencia. El StandardScaler solo no arregla esto.
<code>ValueError: Input contains NaN</code>	Quedaron nulos de antes. Hay que limpiar antes de normalizar.
Aparecen infinitos después de dividir	Alguna central tiene potencia cero o falta el dato.

Tres preguntas de repaso

1. Por qué hacen falta dos normalizaciones

Resuelven problemas distintos. Dividir por la potencia saca el tamaño de la central, que es una decisión del negocio. El StandardScaler iguala el peso de las veinticuatro horas entre sí, que es un requisito del algoritmo.

2. Qué es el factor de planta horario

La generación de esa hora dividida por la potencia instalada. Va entre cero y uno y permite comparar una central de 480 MW con una de 45.

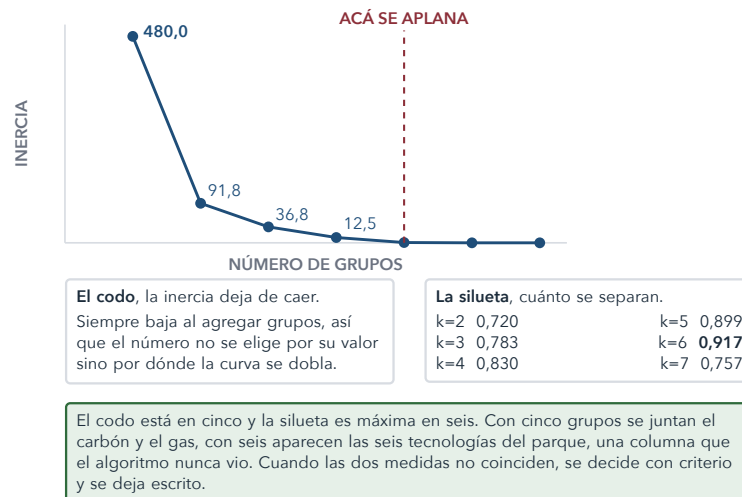
3. Qué pasa si te saltas la primera normalización

Las distancias quedan dominadas por el tamaño. En el bloque 7 se mide, y el índice de Rand ajustado cae de 1,000 a 0,324.

3. Bloque 2, K-Means

Qué se vio

K-Means pide el número de grupos por adelantado, y con cualquier número devuelve un resultado que se ve razonable. Así que el bloque es sobre cómo se elige ese número.



El código, paso a paso

2.1 Cuatro grupos

```
from sklearn.cluster import KMeans

# K-Means pide el número de grupos de entrada. Ese es su límite y su decisión.
km = KMeans(n_clusters=4, random_state=2026, n_init=10).fit(X)

print(pd.Series(km.labels_, index=factor.index).sort_values().to_string())
```

```
central
Carboeléctrica Costa Brava      0
Termoeléctrica Bahía Norte      0
Termoeléctrica Puerto Sur       0
Central Vega Azul               0
Central Tres Saltos             0
Termoeléctrica Valle Central    0
Central Río Manso Alto          0
Central Aguas Claras            0
Carboeléctrica Roca Gris        0
Central Salto Verde             0
Parque Solar Altiplano          1
... [y siguen las diez restantes]
```

Las etiquetas son números, no nombres. Ponerles nombre es trabajo tuyo. El `random_state` fija la semilla y `n_init` prueba diez puntos de partida y se queda con el mejor.

2.2 Contra la tecnología

```
# La tabla que importa. Los grupos que encontró contra la tecnología real,
# que el algoritmo nunca vio.
tecnologia = centrales.set_index("central")["tecnologia"]

print(pd.crosstab(tecnologia[factor.index], km.labels_).to_string())
```

```
col_0      0  1  2  3
tecnologia
carbon      2  0  0  0
diesel      0  0  0  1
eolica      0  0  4  0
gas         3  0  0  0
hidro       5  0  0  0
solar       0  5  0  0
```

Un grupo junta carbón, gas e hidro. Las tres son de base y generan parejo todo el día, así que por forma del perfil son la misma cosa.

2.3 La inercia

```
# Cómo se elige el número de grupos. La inercia es lo que queda sin explicar,
# y siempre baja al agregar grupos. Se busca el codo, donde deja de bajar fuerte.
inercias = []
for k in range(1, 9):
    inercias.append(KMeans(n_clusters=k, random_state=2026, n_init=10).fit(X).inertia_)

for k, v in zip(range(1, 9), inercias):
    print(k, round(v, 1))
```

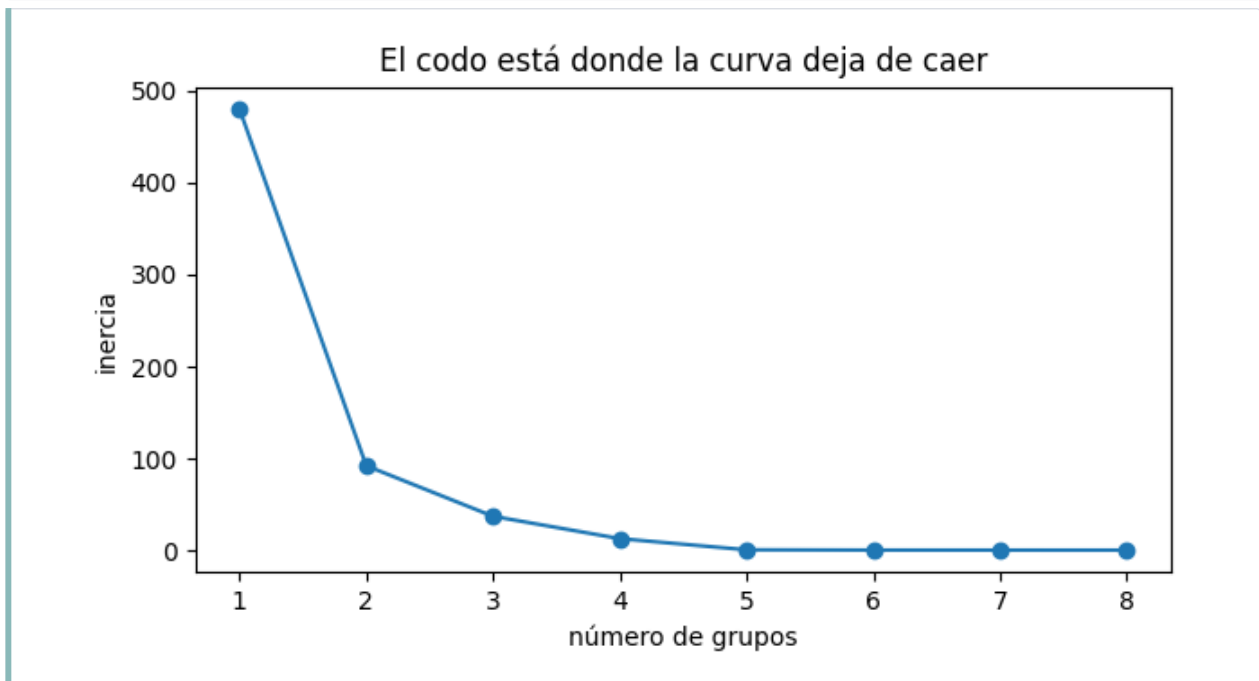
```
1 480.0
2 91.8
3 36.8
4 12.5
5 0.4
6 0.1
7 0.0
8 0.0
```

La inercia es lo que queda sin explicar y siempre baja al agregar grupos, así que el número no se elige por el valor sino por dónde la curva se dobla.

2.4 El codo, dibujado

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(range(1, 9), inercias, marker="o")
ax.set_title("El codo está donde la curva deja de caer")
ax.set_xlabel("número de grupos")
ax.set_ylabel("inercia")
plt.show()
```



Cae en picada hasta tres y desde cinco es plana. El codo casi nunca es un punto nítido, es una zona.

2.5 Seis grupos

```
# Con seis grupos pasa algo que vale la pena mirar.
km6 = KMeans(n_clusters=6, random_state=2026, n_init=10).fit(X)

print(pd.crosstab(tecnologia[factor.index], km6.labels_).to_string())
```

```
col_0      0  1  2  3  4  5
tecnologia
carbon      0  0  0  0  0  2
diesel      0  0  0  1  0  0
eolica      0  0  4  0  0  0
gas         0  0  0  0  3  0
hidro       5  0  0  0  0  0
solar       0  5  0  0  0  0
```

Las seis tecnologías, cada una en su grupo, sin una sola mezcla. Nadie le contó al algoritmo que existían seis tecnologías ni cuáles eran.

2.6 Y sin normalizar

```
# Y ahora la prueba que justifica todo el bloque 1. Lo mismo, pero sin haber
# dividido por la potencia instalada.
X_crudo = StandardScaler().fit_transform(perfil)
km_crudo = KMeans(n_clusters=6, random_state=2026, n_init=10).fit(X_crudo)

print(pd.crosstab(tecnologia[perfil.index], km_crudo.labels_).to_string())
```

```
col_0      0  1  2  3  4  5
tecnologia
carbon      0  0  1  0  1  0
diesel      0  0  0  0  0  1
eolica      3  0  0  0  0  1
gas         0  1  0  1  1  0
hidro       1  1  0  2  1  0
solar       0  0  0  0  0  5
```

El mismo algoritmo, el mismo número de grupos, los mismos datos, pero sin haber dividido por la potencia. Se desarma. Las eólicas se reparten en dos grupos, la hidro en cuatro y el carbón queda partido.

La solución del ejercicio

El ejercicio pedía devolver los centros al espacio original.

```
centros = escalador.inverse_transform(km6.cluster_centers_)

print(pd.DataFrame(centros, columns=factor.columns).iloc[:, 10:15].round(2).to_string())
```

Quedan seis filas de veinticuatro columnas, y cada una es el día tipo de su grupo. Ahí se ve que el grupo solar tiene la campana del mediodía y el grupo diésel el bloque de la noche. Es la mejor forma de ponerle nombre a un grupo.

Referencia rápida

Instrucción	Para qué sirve
KMeans(n_clusters=, random_state=, n_init=)	Agrupar en un número fijo de grupos
.labels_	El grupo de cada fila, como número
.inertia_	Lo que queda sin explicar
.cluster_centers_	El centro de cada grupo, en el espacio escalado
escalador.inverse_transform(c)	Devuelve los centros al espacio original
.predict(nuevo)	Asigna filas nuevas a los grupos ya definidos

Tres preguntas de repaso

1. Cuál es el límite principal de K-Means

Que pide el número de grupos por adelantado y con cualquier número entrega algo que se ve razonable.

2. Por qué no se elige el número por el valor de la inercia

Porque la inercia siempre baja al agregar grupos. Con tantos grupos como puntos, da cero. Lo que se busca es el codo.

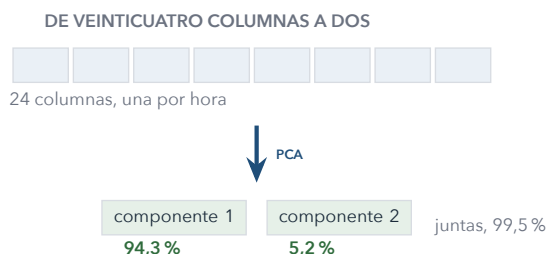
3. Qué demuestra la celda sin normalizar

Que la preparación del bloque 1 no era un trámite. El mismo algoritmo sobre los mismos datos sin normalizar reparte las tecnologías entre grupos sin sentido.

4. Bloque 3, PCA

Qué se vio

Una tabla de veinticuatro columnas no se puede dibujar, y sin dibujarla es muy difícil convencer a alguien de que los grupos son reales. PCA busca las direcciones donde los datos más se estiran y las ordena.



PCA no agrupa, reduce. Busca las direcciones en las que los datos más se estiran y las ordena de mayor a menor. Acá la primera dirección se lleva el 94,3 por ciento de la variación, así que casi toda la diferencia entre las veinte centrales cabe en un solo número. Con dos ya se puede dibujar en una hoja.

Lo que se pierde. Las componentes no tienen nombre. La primera no es la hora doce ni el turno de noche, es una mezcla de las veinticuatro horas con un peso cada una. Se puede mirar esos pesos para interpretarla, y a veces no se parece a nada que uno pueda contar en una reunión. Ese es el precio de reducir.

El código, paso a paso

3.1 La varianza explicada

```
from sklearn.decomposition import PCA

# PCA busca las direcciones donde los datos más se estiran, y las ordena.
pca = PCA().fit(X)

print("cuánto explica cada componente")
print(pd.Series(pca.explained_variance_ratio_[0:6]).round(3).to_string())
```

```
cuánto explica cada componente
0    0.943
1    0.052
2    0.005
3    0.000
4    0.000
5    0.000
```

94,3 por ciento en la primera componente. Con datos reales este número es más chico, típicamente entre 40 y 70 por ciento. Acá es alto porque las centrales del curso son muy distintas entre tecnologías y muy parecidas dentro de cada una.

3.2 La acumulada

```
# Con dos componentes ya está casi todo. Eso es lo que permite dibujarlo.
acumulada = pca.explained_variance_ratio_.cumsum()

print("con 1 componente ", round(acumulada[0], 3))
print("con 2 componentes", round(acumulada[1], 3))
print("con 3 componentes", round(acumulada[2], 3))
```

```
con 1 componente  0.943
con 2 componentes 0.995
con 3 componentes 1.0
```

Con dos componentes, 99,5 por ciento. La regla informal es quedarse con las que suman 80 o 90.

3.3 Las centrales en dos dimensiones

```
# Las veinte centrales en dos dimensiones, coloreadas por tecnología.
Z = PCA(n_components=2).fit_transform(X)
mapa = pd.DataFrame(Z, columns=["pc1", "pc2"], index=factor.index)
mapa["tecnologia"] = tecnologia[mapa.index].values

print(mapa.groupby("tecnologia")[["pc1", "pc2"]].mean().round(2).to_string())
```

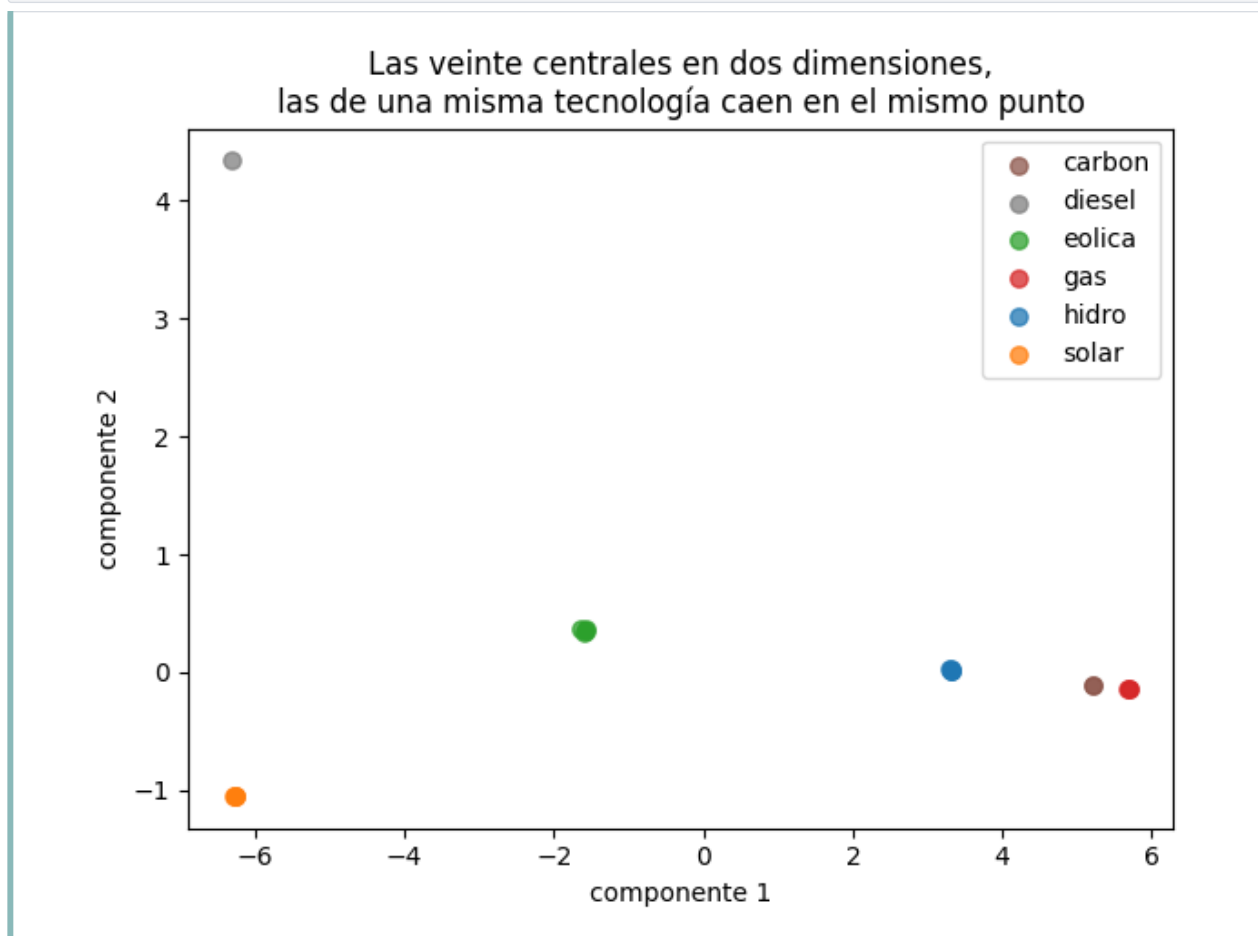
	pc1	pc2
tecnologia		
carbon	5.21	-0.11
diesel	-6.30	4.34
eolica	-1.59	0.36
gas	5.69	-0.14
hidro	3.31	0.02
solar	-6.27	-1.05

Cada central queda con dos coordenadas. El signo no significa nada por sí mismo, PCA podría entregar la misma solución con los ejes dados vuelta. Lo que importa es quién queda lejos de quién.

3.4 El gráfico

```
COLORES = {"hidro": "tab:blue", "solar": "tab:orange", "eolica": "tab:green",
           "gas": "tab:red", "carbon": "tab:brown", "diesel": "tab:gray"}

# Son veinte puntos, pero se ven seis. Las centrales de una misma tecnología tienen casi el
# mismo perfil normalizado y caen una encima de la otra.
fig, ax = plt.subplots(figsize=(7, 5))
for t, g in mapa.groupby("tecnologia"):
    ax.scatter(g["pc1"], g["pc2"], label=t, s=45, alpha=0.75, color=COLORES[t])
ax.set_title("Las veinte centrales en dos dimensiones,\n"
            "las de una misma tecnología caen en el mismo punto")
ax.set_xlabel("componente 1")
ax.set_ylabel("componente 2")
ax.legend()
plt.show()
```



Las seis tecnologías separadas a simple vista, con la solar y el diésel en extremos opuestos. Son veinte centrales y se ven seis puntos, porque las de una misma tecnología tienen casi el mismo perfil normalizado y caen una encima de la otra. Con datos reales se verían nubes, no puntos. Esta es la figura que uno lleva a una reunión.

3.5 Qué es la componente 1

```
# Qué es la componente 1. Se mira su peso en cada una de las 24 horas.  
pesos = pd.Series(PCA(n_components=2).fit(X).components_[0], index=factor.columns)  
  
print(pesos.round(2).to_string())
```

```
hora  
0    0.21  
1    0.21  
2    0.21  
3    0.21  
4    0.21  
5    0.21  
6    0.21  
7    0.21  
8    0.21  
9    0.21  
10   0.20  
11   0.20  
12   0.20  
13   0.20  
14   0.20  
15   0.21  
16   0.21  
17   0.21  
18   0.19  
19   0.19  
20   0.19  
21   0.19  
22   0.19  
23   0.21
```

Veinticuatro pesos, todos entre 0,19 y 0,21, o sea que la primera componente suma todas las horas casi por igual y mide cuánto genera la central en promedio a lo largo del día, en proporción a su potencia. Por eso separa las que generan parejo todo el día, el carbón, el gas y la hidro, de las que no, la solar y el diésel. Cuando los pesos son parecidos, la componente es un promedio. Cuando unos son positivos y otros negativos, es un contraste.

La solución del ejercicio

El ejercicio pedía mirar la tercera componente.

```
p3 = PCA(n_components=3).fit(X)  
  
print(p3.explained_variance_ratio_.round(4))
```

La tercera agrega 0,5 por ciento. La respuesta es que no vale la pena dibujarla, y es una decisión que se toma mirando el número y no por costumbre.

Referencia rápida

Instrucción	Para qué sirve
<code>PCA()</code> sin argumentos	Todas las componentes, para mirar la varianza
<code>PCA(n_components=2)</code>	Reduce a dos dimensiones
<code>.explained_variance_ratio_</code>	Cuánto explica cada componente
<code>.cumsum()</code>	La varianza acumulada
<code>.components_</code>	Los pesos de cada variable en cada componente
<code>.fit_transform(X)</code>	Ajusta y devuelve las coordenadas nuevas

Tres preguntas de repaso

1. PCA agrupa

No. Es un cambio de coordenadas. Los grupos que se ven en el gráfico son los que encontró K-Means.

2. Hay que normalizar antes de PCA

Sí, siempre. PCA busca direcciones de máxima varianza y una columna con números grandes se lleva la primera componente sin merecerlo.

3. Qué se pierde al reducir

El nombre. Las componentes son mezclas de las variables originales y no corresponden a nada que uno pueda contar en una reunión sin trabajo adicional.

5. Bloque 4, DBSCAN

Qué se vio

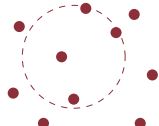
K-Means obliga a que todo punto caiga en algún grupo. DBSCAN arma grupos por densidad y deja fuera lo que no alcanza. A eso lo llama ruido, y la palabra engaña.

UN PUNTO ENTRA A UN GRUPO SI TIENE CINCO VECINOS CERCA



DÍAS DE UNA SOLAR

Cada día se parece al anterior. Se forman grupos densos.



DÍAS DE UNA EÓLICA

Ningún día repite. No hay cinco vecinos dentro del radio.

radio	ruido
1,5	1.464
2,0	1.462
2,5	1.104
3,0	5

1.464 son exactamente las cuatro eólicas por los 366 días del año.

El ruido de DBSCAN no es basura. Acá dice cuáles son las centrales cuya producción no tiene un día tipo, que es una conclusión de negocio y no un descarte técnico. Subir el radio a 3,0 hace desaparecer el ruido y también el hallazgo.

El código, paso a paso

4.1 Otra unidad de análisis

```
# Cambiamos de unidad. Ya no una fila por central, sino una por central y día.
perfil_dia = gen.pivot_table(index=["central", "fecha"], columns="hora",
                             values="mwh", aggfunc="mean")
potencia_dia = perfil_dia.index.get_level_values("central").map(potencia)
factor_dia = perfil_dia.div(potencia_dia, axis=0)
X_dia = StandardScaler().fit_transform(factor_dia)

print(X_dia.shape, "veinte centrales por 366 días")
```

```
(7320, 24) veinte centrales por 366 días
```

Ya no una fila por central, sino una por central y por día. Veinte por 366 son 7.320 filas.

4.2 La curva del quinto vecino

```
from sklearn.neighbors import NearestNeighbors
import numpy as np

# DBSCAN no pide número de grupos, pide un radio. Para elegirlo se mira a qué
# distancia está el quinto vecino de cada punto, ordenado de menor a mayor.
distancias, _ = NearestNeighbors(n_neighbors=5).fit(X_dia).kneighbors(X_dia)
quinto = np.sort(distancias[:, -1])

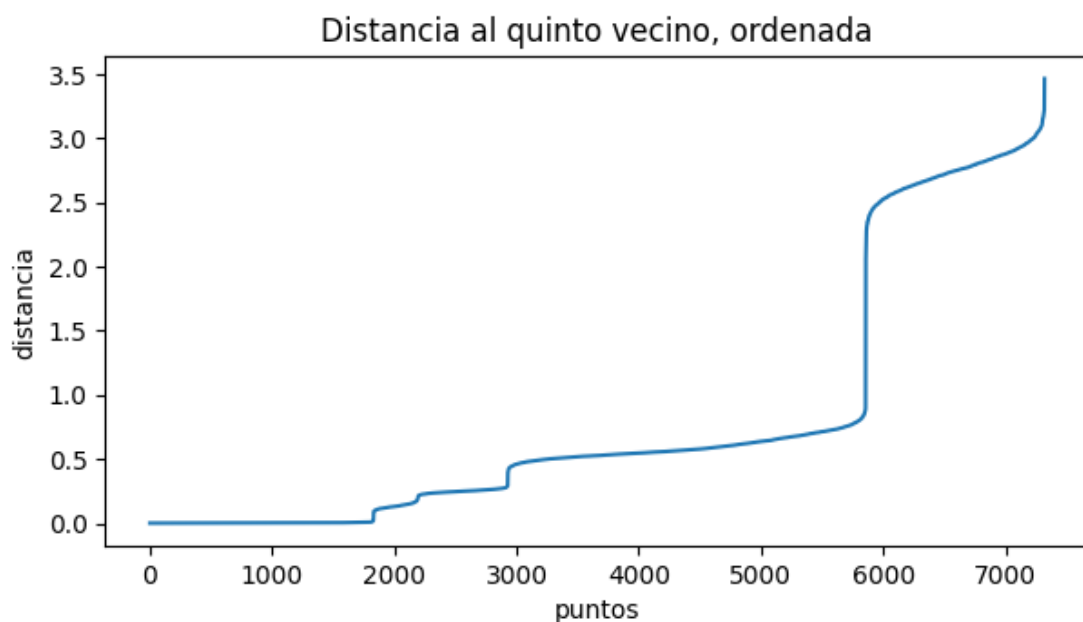
for p in (50, 75, 90, 95, 99):
    print(f"percentil {p}: {np.percentile(quinto, p):.2f}")
```

```
percentil 50: 0.53
percentil 75: 0.71
percentil 90: 2.75
percentil 95: 2.86
percentil 99: 3.02
```

DBSCAN no pide grupos, pide un radio. Para elegirlo se mira a qué distancia está el quinto vecino de cada punto. La mediana es 0,53 y el percentil 90 salta a 2,75, o sea que hay dos poblaciones, una densa y una dispersa.

4.3 El gráfico del codo

```
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(quinto)
ax.set_title("Distancia al quinto vecino, ordenada")
ax.set_xlabel("puntos")
ax.set_ylabel("distancia")
plt.show()
```



La misma información dibujada. Plano hasta el 80 por ciento de los puntos y después una pared.

4.4 Cuatro radios

```
from sklearn.cluster import DBSCAN

# El salto de la curva está entre 0,7 y 2,7. Probamos tres radios dentro del salto y uno
# por encima.
for eps in (1.5, 2.0, 2.5, 3.0):
    db = DBSCAN(eps=eps, min_samples=5).fit(X_dia)
    grupos = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
    ruido = int((db.labels_ == -1).sum())
    print(f"eps {eps}: {grupos} grupos, {ruido} puntos de ruido"
          f" ({100 * ruido / len(X_dia):.1f} por ciento)")
```

```
eps 1.5: 3 grupos, 1464 puntos de ruido (20.0 por ciento)
eps 2.0: 3 grupos, 1462 puntos de ruido (20.0 por ciento)
eps 2.5: 15 grupos, 1104 puntos de ruido (15.1 por ciento)
eps 3.0: 3 grupos, 5 puntos de ruido (0.1 por ciento)
```

Con radio 1,5 quedan 1.464 puntos fuera de todo grupo. Con 3,0 quedan cinco. Tres de los radios caen dentro del salto de la curva y uno por encima.

Fíjate en el 2,5, que arma 15 grupos. No son quince tipos de día. Ese radio cae en el borde alto del salto, donde empiezan las distancias de los días eólicos, y en esa zona mover un poco el radio parte y junta grupos. Por eso el radio se elige lejos del salto, donde moverlo no cambia el resultado.

4.5 Quiénes son

```
# Los 15 grupos del radio 2,5 no son un hallazgo. Ese radio cae en el borde alto del salto,
# donde empiezan las distancias de los días eólicos, y ahí moverlo un poco parte y junta
# grupos. Por eso el radio se elige lejos del salto, donde moverlo no cambia el resultado.

# Con radio 1,5 quedan 1.464 puntos fuera de todo grupo. Quiénes son.
db = DBSCAN(eps=1.5, min_samples=5).fit(X_dia)
tecnologia_dia = perfil_dia.index.get_level_values("central").map(tecnologia)

print(pd.crosstab(tecnologia_dia, db.labels_).to_string())
```

col_0	-1	0	1	2
row_0				
carbon	0	732	0	0
diesel	0	0	366	0
eolica	1464	0	0	0
gas	0	1098	0	0
hidro	0	1830	0	0
solar	0	0	0	1830

El ruido son las cuatro eólicas completas, y ninguna otra tecnología tiene un solo punto marcado. Los tres grupos que sí armó son las mismas familias del bloque 2.

4.6 La cuenta

```
# Mil cuatrocientos sesenta y cuatro son cuatro centrales por 366 días.
print("días eólicos en la tabla", int((tecnologia_dia == "eolica").sum()))
print("puntos marcados como ruido", int((db.labels_ == -1).sum()))
```

```
días eólicos en la tabla 1464
puntos marcados como ruido 1464
```

1.464 días eólicos en la tabla, 1.464 puntos de ruido. Exacto, no aproximado. Ningún día de un parque eólico tiene cinco vecinos dentro del radio, porque el viento no repite forma de un día a otro.

La solución del ejercicio

El ejercicio pedía subir el radio.

```
prueba = DBSCAN(eps=3.5, min_samples=5).fit(X_dia)

print("grupos", len(set(prueba.labels_)) - (1 if -1 in prueba.labels_ else 0),
      " ruido", int((prueba.labels_ == -1).sum()))
```

Con radio grande el ruido desaparece y también el hallazgo. Cuál radio usar depende de para qué es el análisis, y esa decisión hay que escribirla.

EL RUIDO NO SIEMPRE ES UN ERROR

DBSCAN dejó fuera de todo grupo a los 1.464 días eólicos. No es una falla del algoritmo.

Un punto queda en un grupo si tiene al menos cinco vecinos dentro del radio. El día de un parque eólico casi nunca se parece a otro día, porque el viento no repite forma. Los días de una solar o de una térmica sí se parecen entre ellos.

O sea que el ruido de DBSCAN acá es un hallazgo. Dice cuáles son las centrales cuya producción no tiene un día tipo, que es una conclusión de negocio y no un descarte técnico.

Referencia rápida

Instrucción	Para qué sirve
DBSCAN(eps=, min_samples=)	Agrupar por densidad, sin número de grupos
.labels_	El grupo de cada punto. Menos uno es ruido
NearestNeighbors(n_neighbors=5)	Para elegir el radio
.kneighbors(X)	Distancias a los vecinos más cercanos
np.percentile(v, p)	Los percentiles de esas distancias

Errores comunes

Lo que aparece en pantalla	Qué pasó
Todo queda como ruido	El radio es demasiado chico para la escala de los datos.
Un solo grupo con todo adentro	El radio es demasiado grande.
ValueError al calcular la silueta	La silueta no acepta la etiqueta menos uno. Hay que sacar el ruido antes.
No existe predict	DBSCAN no asigna puntos nuevos. Es de una sola pasada.

Tres preguntas de repaso

1. Qué pide DBSCAN en vez del número de grupos

Un radio y un mínimo de vecinos. El radio se elige mirando la distancia al quinto vecino y buscando el salto de la curva.

2. Por qué los días eólicos quedan como ruido

Porque el viento no repite forma de un día a otro, así que ningún día eólico tiene cinco vecinos dentro del radio.

3. Qué se pierde al subir el radio hasta que el ruido desaparece

El hallazgo. Con radio 3,0 todo cae en un grupo y ya no se sabe cuáles centrales no tienen un día tipo.

6. Bloque 5, temas en el texto libre

Qué se vio

La bitácora del Módulo 4 trae una columna de observación escrita a mano, 600 textos cortos que nadie etiquetó. LDA los lee y propone temas, que son grupos de palabras que tienden a aparecer juntas.

El código, paso a paso

5.1 La columna que faltaba

```
# La bitácora del Módulo 4, ahora con la columna que no habíamos usado.
mant = pd.read_csv("mantenimiento.csv", parse_dates=["fecha"])

muestra = mant[["tipo_evento", "observacion"]].head(3).copy()
muestra["observacion"] = muestra["observacion"].str.slice(0, 58) + "..."

print(muestra.to_string(index=False))
```

tipo_evento	observacion
preventivo	Se registra la lubricación periódica sobre el grupo hidráulico...
preventivo	El operador reporta la lubricación periódica, se revisa el...
falla_mecanica	El operador reporta vibración fuera de norma, se revisa el...

El vocabulario es distinto según el tipo de evento. Eso es lo que LDA va a aprovechar, aunque no sepa que existen los tipos.

5.2 Las palabras vacías

```
from sklearn.feature_extraction.text import CountVectorizer

# Las palabras que aparecen en todas partes no distinguen nada, se sacan.
VACIAS = ["el", "la", "los", "las", "un", "una", "de", "del", "en", "y", "se",
          "que", "por", "con", "sobre", "tras", "su", "sus", "al", "lo", "es",
          "durante", "ronda", "turno", "registra", "detecta", "reporta",
          "evento", "operador", "revisa"]
vectorizador = CountVectorizer(stop_words=VACIAS, max_df=0.9, min_df=3)
matriz = vectorizador.fit_transform(mant["observacion"])

print(matriz.shape, "seiscientos textos por", len(vectorizador.get_feature_names_out()),
      ↪ "palabras")
```

```
(600, 151) seiscientos textos por 151 palabras
```

La lista tiene dos partes. Las palabras funcionales del español, el, la, en. Y palabras propias de este dominio que aparecen en todos los textos, turno, registra, reporta. Las segundas hay que descubrirlas mirando el corpus, no vienen en ninguna lista estándar.

5.3 Los seis temas

```
from sklearn.decomposition import LatentDirichletAllocation

# LDA busca temas. No sabe que existen seis tipos de evento, le pedimos seis temas.
lda = LatentDirichletAllocation(n_components=6, random_state=2026).fit(matriz)
palabras = vectorizador.get_feature_names_out()

for i, tema in enumerate(lda.components_):
    top = [palabras[j] for j in tema.argsort()[-7:][::-1]]
    print(f"tema {i}:", ", ".join(top))
```

```
tema 0: estructura, reaprietan, sella, patio, nieve, acumulación, inspecciona
tema 1: vibración, completa, pauta, observaciones, acoplamiento, sin, conexiones
tema 2: servicio, orden, trabajo, pendiente, devuelve, repara, despacho
tema 3: filtros, rutina, lectura, ajuste, sin, cambio, cambian
tema 4: tensión, interruptor, protección, relé, elevada, corriente, fuga
tema 5: aceite, eje, rodamiento, programa, detalle, revisión, verificación
```

El tema 0 habla de estructura, nieve y patio. El 4 de tensión, interruptor y relé. Se entienden sin que nadie los explique. El número seis lo pusiste tú.

5.4 Contra el tipo de evento

```
# Y ahora la prueba. A cada texto se le asigna su tema dominante, y se cruza
# contra el tipo de evento, que LDA nunca vio.
dominante = lda.transform(matriz).argmax(axis=1)
cruce = pd.crosstab(mant["tipo_evento"], dominante)

print(cruce.to_string())
```

col_0	0	1	2	3	4	5
tipo_evento						
correctivo	0	0	118	0	0	0
evento_climatico	80	0	0	0	0	0
falla_electrica	0	0	0	0	47	0
falla_mecanica	0	40	0	0	0	58
inspeccion	0	0	0	25	8	66
preventivo	0	73	0	85	0	0

A cada texto se le asigna su tema dominante y se cruza contra el tipo, que LDA nunca vio. Los correctivos caen todos en un tema y los climáticos todos en otro. Las inspecciones se reparten entre tres temas y los preventivos entre dos, porque comparten vocabulario de rutina y pauta.

5.5 La cuenta

```
# Para cada tipo de evento, cuántos textos cayeron en su tema más frecuente, sumado sobre los
# seis tipos. No es un acierto uno a uno, un mismo tema puede ser el más frecuente de dos tipos.
print("textos en el tema mayoritario de su tipo", int(cruce.max(axis=1).sum()), "de", len(mant))
```

```
textos en el tema mayoritario de su tipo 454 de 600
```

454 de 600 caen en el tema mayoritario de su tipo. La cuenta toma, para cada tipo, su tema más frecuente, y un mismo tema puede ser el más frecuente de dos tipos, como pasa con la inspección y la falla mecánica. No es una exactitud, porque LDA no estaba tratando de predecir el tipo. Es una medida de cuánto se parecen los temas que emergieron a las categorías que ya existían.

La solución del ejercicio

El ejercicio pedía tres temas en vez de seis.

```
lda3 = LatentDirichletAllocation(n_components=3, random_state=2026).fit(matriz)

print(pd.crosstab(mant["tipo_evento"], lda3.transform(matriz).argmax(axis=1)).to_string())
```

Se juntan los de rutina por un lado y los de falla por otro. Es una agrupación razonable y distinta de la oficial. El número de temas define qué nivel de detalle estás mirando.

Referencia rápida

Instrucción	Para qué sirve
CountVectorizer(stop_words=, min_df=)	Cuenta palabras y descarta las raras
.get_feature_names_out()	Los nombres de las columnas de palabras
LatentDirichletAllocation(n_components=)	Busca temas en los textos
.components_	El número de temas lo eliges tú
.transform(m).argmax(axis=1)	El peso de cada palabra en cada tema
	El tema dominante de cada texto

Tres preguntas de repaso

1. LDA entiende el texto

No. Cuenta palabras y busca grupos de palabras que aparecen juntas. No sabe de orden ni de significado.

2. De dónde sale la lista de palabras vacías

Parte es estándar del español y parte se arma mirando el corpus. Elegir mal cambia los temas.

3. Dónde se equivoca el LDA acá

En las inspecciones y los preventivos, que comparten vocabulario de rutina. Eso también es información, dice qué categorías son parecidas en cómo se escriben.

7. Bloque 6, evaluar sin respuesta correcta

Qué se vio

Cuatro agrupamientos y ninguna respuesta contra la cual compararse. Esa es la situación normal y es lo que hace incómodo a este tipo de métodos.

El código, paso a paso

6.1 La silueta

```
from sklearn.metrics import silhouette_score

# La silueta mide, para cada punto, qué tan cerca está de su grupo comparado
# con el grupo vecino. Va de menos uno a uno, y no necesita la respuesta.
for k in range(2, 8):
    etiquetas = KMeans(n_clusters=k, random_state=2026, n_init=10).fit_predict(X)
    print(f"k={k} silueta {silhouette_score(X, etiquetas):.3f}")
```

```
k=2 silueta 0.720
k=3 silueta 0.783
k=4 silueta 0.830
k=5 silueta 0.899
k=6 silueta 0.917
k=7 silueta 0.757
```

Mide, para cada punto, qué tan cerca está de su grupo comparado con el grupo vecino más próximo. Va de menos uno a uno y no necesita la respuesta. Acá el máximo está en seis grupos.

6.2 Las dos medidas juntas

```
# La silueta y el codo no siempre apuntan al mismo número. Acá hay que mirarlos
# juntos, con la inercia al lado.
resumen = pd.DataFrame({"inercia": [round(v, 1) for v in inercias[1:7]],
                        "silueta": [round(silhouette_score(
                            X, KMeans(n_clusters=k, random_state=2026,
                                n_init=10).fit_predict(X)), 3)
                                for k in range(2, 8)]},
                        index=range(2, 8))

print(resumen.to_string())
```

```
inercia silueta
2      91.8    0.720
3      36.8    0.783
4      12.5    0.830
5       0.4    0.899
6       0.1    0.917
7       0.0    0.757
```

Acá no apuntan al mismo número. El codo está en cinco, donde la inercia cae casi a cero, y la silueta es máxima en seis. Cuando coinciden uno está cómodo. Cuando no, se muestran las dos y se decide con criterio, y eso se escribe en el informe. La diferencia entre las dos está en la celda siguiente.

6.3 Cinco grupos también sirve

```
# Y la prueba que en el trabajo real no existe, comparar contra la tecnología.
km5 = KMeans(n_clusters=5, random_state=2026, n_init=10).fit(X)

print(pd.crosstab(tecnologia[factor.index], km5.labels_).to_string())
```

```
col_0      0  1  2  3  4
tecnologia
carbon      0  0  0  0  2
diesel      0  0  0  1  0
eolica      0  0  4  0  0
gas         0  0  0  0  3
hidro       5  0  0  0  0
solar       0  5  0  0  0
```

Con cinco grupos, carbón y gas quedan juntos. No está mal, las dos son térmicas de base y su día se parece. La diferencia entre cinco y seis es si te importa distinguirlas, que es una pregunta de negocio y la silueta no la puede responder.

La solución del ejercicio

El ejercicio pedía la silueta de DBSCAN.

```
db3 = DBSCAN(eps=3.0, min_samples=5).fit(X_dia)
sin_ruido = db3.labels_ != -1

print(round(silhouette_score(X_dia[sin_ruido], db3.labels_[sin_ruido]), 3))
```

Lo primero que descubres es que hay que sacar el ruido, porque la silueta no acepta la etiqueta menos uno. Eso ya es una lección, estas medidas suponen que todo punto está en un grupo.

Referencia rápida

Instrucción	Para qué sirve
<code>silhouette_score(X, etiquetas)</code>	Qué tan separados quedaron los grupos
<code>calinski_harabasz_score</code>	Otra medida interna, suele coincidir
<code>davies_bouldin_score</code>	Otra más. Acá menos es mejor
<code>adjusted_rand_score(verdad, etiq)</code>	Solo si tienes la respuesta correcta

Tres preguntas de repaso

1. Qué mide la silueta

Para cada punto, qué tan cerca está de su propio grupo comparado con el grupo vecino. No necesita la respuesta correcta.

2. Una silueta alta garantiza que el agrupamiento sirve

No. Garantiza que los grupos están separados en el espacio en que los mediste. Si ese espacio no representa lo que importa, los grupos no sirven.

3. Qué hacer si no puedes nombrar un grupo

Volver atrás. O las variables no son las correctas, o la normalización está mal, o el número de grupos no es ese.

8. Bloque 7, contra la verdad

Qué se vio

El parque tiene una columna de tecnología y ningún algoritmo del módulo la vio. Eso permite hacer la pregunta que en el trabajo real no se puede.

lo que hizo el algoritmo	índice de Rand
K-Means con 6 grupos, normalizado	1,000
K-Means con 4 grupos, normalizado	0,568
K-Means con 6 grupos, sin normalizar	0,324

Uno quiere decir que el agrupamiento reproduce exactamente la tecnología, y cero que no tiene nada que ver con ella. Con seis grupos y los datos normalizados el algoritmo recuperó las seis tecnologías sin haberlas visto nunca. Sin normalizar por potencia, el mismo algoritmo sobre los mismos datos baja a 0,324, que es poco más que azar.

Y ahora la advertencia. Este cuadro existe porque tenemos la columna de tecnología. En el trabajo real no la vas a tener, y por eso el bloque 6 enseña a evaluar sin ella. El índice de Rand es del laboratorio, la silueta es de la vida real.

El código, paso a paso

7.1 Los tres índices

```
# Los tres agrupamientos, contra la tecnología real.
from sklearn.metrics import adjusted_rand_score

verdad = tecnologia[factor.index].values
for nombre, etiquetas in [("K-Means k=4", km.labels_),
                          ("K-Means k=6", km6.labels_),
                          ("K-Means sin normalizar", km_crudo.labels_)]:
    print(f"{nombre:26s} índice de Rand ajustado {adjusted_rand_score(verdad, etiquetas):.3f}")
```

```
K-Means k=4           índice de Rand ajustado 0.568
K-Means k=6           índice de Rand ajustado 1.000
K-Means sin normalizar índice de Rand ajustado 0.324
```

Uno quiere decir coincidencia exacta y cero quiere decir tan bueno como repartir al azar. El índice está ajustado justamente para que el azar dé cero.

Un uno exacto acá, y una sospecha en el trabajo. El 1,000 de la segunda línea sale porque los datos son sintéticos y la tecnología determina por completo el perfil de cada central. Acá es lo esperable. Con datos reales un uno sería motivo de sospecha, no de celebración, y lo primero que hay que revisar es si hubo una fuga de información, es decir si alguna variable de entrada traía la respuesta adentro. El aviso está también dentro del notebook, en la celda que sigue.

7.2 La tabla, otra vez

```
# Uno quiere decir que el agrupamiento reproduce exactamente la tecnología,
# y cero que no tiene nada que ver con ella.
print("con seis grupos y normalizando, el algoritmo recuperó las seis tecnologías")
print(pd.crosstab(tecnologia[factor.index], km6.labels_).to_string())
```

```
con seis grupos y normalizando, el algoritmo recuperó las seis tecnologías
col_0      0  1  2  3  4  5
tecnologia
carbon      0  0  0  0  0  2
diesel      0  0  0  1  0  0
eolica      0  0  4  0  0  0
gas         0  0  0  0  3  0
hidro       5  0  0  0  0  0
solar       0  5  0  0  0  0
```

Seis grupos, seis tecnologías, cero mezclas. Y la diferencia con el 0,324 de la fila sin normalizar es una sola línea de código, la de dividir por la potencia.

La solución del ejercicio

El ejercicio pedía el índice de Rand sobre el LDA.

```
print(round(adjusted_rand_score(mant["tipo_evento"], dominante), 3))
```

Da bastante más bajo, porque los temas no se alinean uno a uno con los tipos y porque el texto es más ambiguo que un perfil horario. Un índice bajo no significa que el método falló, puede significar que la categoría oficial no era la única forma razonable de agrupar.

Tres preguntas de repaso

1. Por qué no usar siempre el índice de Rand

Porque necesita la respuesta correcta, y si la tuvieras no estarías agrupando.

2. Qué explica la diferencia entre 1,000 y 0,324

Una línea de código, la de dividir por la potencia instalada.

3. Con datos reales, un índice de 1,000 sería buena noticia

Sería motivo para revisar. Un uno perfecto sugiere que alguna variable de entrada contenía la respuesta.

Hoja de referencia del Módulo 5

Para imprimir y tener al lado

PREPARAR, QUE ES DONDE SE GANA

<code>df.div(potencia, axis=0)</code>	Decisión del negocio. Deja el factor de planta
<code>StandardScaler().fit_transform(df)</code>	Requisito del algoritmo. Iguala las columnas

AGRUPAR

<code>KMeans(n_clusters=, random_state=, n_init=)</code>	Pide el número de grupos
<code>.inertia_</code>	Busca el codo, no el valor
<code>DBSCAN(eps=, min_samples=)</code>	Pide un radio. Menos uno es ruido
<code>NearestNeighbors(n_neighbors=5)</code>	Para elegir el radio, busca el salto

REDUCIR Y DIBUJAR

<code>PCA(n_components=2)</code>	Reduce, no agrupa
<code>.explained_variance_ratio_.cumsum()</code>	Cuánto conservas. Acá 99,5 % con dos
<code>.components_</code>	Los pesos, para interpretar la componente

TEXTO LIBRE

<code>CountVectorizer(stop_words=, min_df=)</code>	Cuenta palabras, descarta las de siempre
<code>LatentDirichletAllocation(n_components=)</code>	Busca temas sin etiquetas previas
	El número de temas lo eliges tú

EVALUAR

<code>silhouette_score(X, etiquetas)</code>	Sin respuesta correcta. Saca el ruido antes
<code>adjusted_rand_score(verdad, etiq)</code>	Solo en el laboratorio, cuando hay respuesta

Las cinco cosas que hay que recordar del módulo

- Normalizar dos veces, una por el negocio y otra por el algoritmo. Ahí se gana.
- K-Means pide el número de grupos y con cualquiera entrega algo razonable.
- PCA reduce, no agrupa, y las componentes no tienen nombre.
- El ruido de DBSCAN puede ser el hallazgo, no el descarte.
- Un grupo que no se puede nombrar no se puede usar para decidir.